

Deployment Strategies

Intermediate Kubernetes

Module 11 of 13



Agenda

Core Strategies

- Rolling updates (default)
- Rollback and revision history
- Blue-green deployments
- Canary deployments

Advanced Topics

- Progressive delivery tooling
- A/B testing & shadow launches
- Deployment safety mechanisms
- EKS best practices & CI/CD

Rolling Updates

The Default Deployment Strategy

How Rolling Updates Work

Incremental Pod Replacement

Kubernetes gradually replaces old ReplicaSet pods with new ones, controlled by two parameters: `maxSurge` and `maxUnavailable`.

`maxSurge`

- Extra pods above desired count
- Absolute number or percentage
- Default: **25%**
- Higher = faster rollout, more resources

`maxUnavailable`

- Pods that can be down during update
- Absolute number or percentage
- Default: **25%**
- Lower = safer but slower rollout



Monitoring Rollout Progress

Tip: Use `--watch` with `kubectl get pods` to see individual pod transitions during a rollout. Combine with `-l app=api-server` to filter.

Pause is powerful: Pausing a rollout mid-way creates a partial canary — some pods run the old version, some the new. Use this for manual verification before resuming.

Rollback and Revision History

Fast Recovery: Rollback reuses the old ReplicaSet — no image pull needed if cached. Recovery takes seconds, not minutes.

Database Caveat: Rollback reverts pods but not database migrations. Always ensure backward-compatible schema changes.



Blue-Green Deployments

Instant Cutover with Full Rollback

Blue-Green Architecture

Core Principle

Run two identical environments in parallel. Route all traffic to one (blue). Deploy changes to the other (green). Switch traffic atomically when ready.

✓ Advantages

- Instant rollback (switch back)
- No mixed versions serving
- Full pre-flight testing

⚠ Trade-offs

- 2x resource cost during deploy
- Database compatibility required
- Session state must be external

Best for: Critical services, compliance-heavy environments, APIs with strict SLAs, and when rollback speed is paramount.



Blue-Green with Ingress / Gateway API

Gateway API advantage: Weight-based routing enables gradual cutover (90/10 → 50/50 → 0/100) rather than all-or-nothing. Blurs the line between blue-green and canary.

Canary Deployments

Controlled Risk with Gradual Traffic Shifting

Canary Concept

Progressive Traffic Shifting

Deploy the new version alongside the current version. Route a small percentage of traffic to it. Observe metrics. Gradually increase traffic or roll back.

Canary Progression

- **Step 1:** 5% traffic to canary
- **Step 2:** Observe error rates, latency
- **Step 3:** Promote to 25%, then 50%
- **Step 4:** Full promotion to 100%

Key Metrics to Watch

- HTTP error rate (5xx responses)
- Request latency (p50, p95, p99)
- Resource consumption (CPU, memory)
- Business metrics (conversion, throughput)



Progressive Delivery: Argo Rollouts

Argo Rollouts replaces Deployments with a custom Rollout CRD. It provides built-in canary steps, analysis templates, and service mesh integration.

Progressive Delivery: Flagger

Key difference: Flagger wraps *existing* Deployments and automates canary analysis based on metrics. No CRD replacement needed.

Advanced Strategies

A/B Testing, Shadow Launches & Feature Flags

Shadow / Dark Launches

Traffic Mirroring

Duplicate live traffic to the new version. The shadow processes requests but its responses are discarded.

Zero risk to users: Shadow receives real traffic patterns but responses are discarded. Validates performance and correctness before any user exposure.

Feature Flags vs Deployment Strategies

Deployment Strategies

- Control at **infrastructure** level
- Entire application version changes
- Operated by platform team
- Rollback = redeploy old version
- Scope: all-or-nothing per pod

Feature Flags

- Control at **application** level
- Toggle individual features in code
- Operated by development team
- Rollback = flip the flag off
- Scope: per-feature, per-user

Recommended pattern: Combine both. Use deployment strategies for infrastructure changes and feature flags for feature activation. Deploy dark (flag off), then enable progressively.



Deployment Safety

Guardrails for Safe Releases

Pod Disruption Budgets

Voluntary disruptions: PDBs protect against node drains, cluster autoscaler, and maintenance operations.

Not for crashes: PDBs do not protect against involuntary disruptions like node failures or OOM kills.

During deployments: PDBs interact with rolling updates. A PDB of `minAvailable: 3` on a 4-replica Deployment effectively forces `maxUnavailable: 1`, regardless of the Deployment's own strategy configuration.



Deployment Conditions & Failure Detection

Stuck rollouts: If a new pod fails readiness probes repeatedly, the rollout stalls. `progressDeadlineSeconds` marks it as failed after the deadline, enabling automated alerting and rollback.

CI/CD Pipeline Integration Patterns

Deployment Pipeline Flow

Build → Test → Image Push → Deploy to Staging → Integration Tests → Deploy → Verify → Promote or Rollback

GitOps principle: Git is the single source of truth. All deployment changes go through Git, enabling audit trails, approvals, and automated rollback to any previous commit.

EKS Deployment Best Practices

Node Management

- Use managed node groups for automatic AMI updates
- Configure PDBs before node group updates
- Use Karpenter for rapid scaling during rollouts

Image Management

- Use ECR with image scanning enabled
- Immutable tags (never reuse :latest)
- Pre-pull images on nodes before rollout

Observability

- CloudWatch Container Insights for metrics
- AWS X-Ray for tracing during canary
- ALB access logs for traffic analysis

EKS tip: When using ALB Ingress Controller for canary deployments, configure `alb.ingress.kubernetes.io/actions.weighted-routing` for precise traffic splitting at the load balancer level.

Strategy Comparison Matrix

Strategy	Downtime	Resource Cost	Rollback Speed	Complexity
Rolling Update	None	Low (+25%)	Medium (new rollout)	Low (built-in)
Blue-Green	None	High (2x)	Instant (switch back)	Low-Medium
Canary	None	Low-Medium	Fast (kill canary)	Medium-High
A/B Testing	None	Medium	Fast (routing change)	High
Shadow	None	High (2x)	N/A (no user impact)	High (mesh required)

Choosing a strategy: Start with rolling updates. Add canary for critical services. Use blue-green when instant rollback is non-negotiable. Layer in feature flags for granular control.

Module Summary

- Rolling updates: zero-downtime with tunable maxSurge/maxUnavailable
- Blue-green: instant rollback via Service selector switching
- Canary: reduced blast radius with gradual traffic shifting
- PDBs protect availability during voluntary disruptions
- progressDeadlineSeconds catches stuck rollouts



Lab 11: Deployment Strategies

Hands-On Exercise

- Configure a rolling update with custom maxSurge and maxUnavailable
- Monitor a rollout and perform a rollback to a specific revision
- Implement a blue-green deployment with Service label switching
- Implement canary deployment with replica ratios
- Set up Pod Disruption Budgets and test with a dry-run node drain
- Implement progressDeadlineSeconds and trigger a failed rollout

Duration: ~45 minutes

Key Takeaways

1. Rolling updates are the baseline. Tune `maxSurge`, `maxUnavailable`, and `minReadySeconds`. Add canary for critical services.

2. Blue-green provides instant rollback. Worth 2x resource cost for critical services. Canary reduces blast radius with metric-driven promotions.

3. Safety mechanisms are not optional. PDBs, `progressDeadlineSeconds`, readiness probes, and `revisionHistoryLimit` should be on every Deployment. Combine with feature flags to decouple deployment from release.

? Questions & Discussion

Module 11: Deployment Strategies

Up Next: Module 12 — Helm & Templating